

平时作业三

2312331 王一诺

一、为什么用时钟边缘（下降沿）做控制信号？

在数字系统中，时钟的作用不是直接表示某个数据值，而是给整个系统提供一个统一的“状态更新时刻”。处理器内部可以粗略分成两类电路：

1. **组合逻辑电路**：例如 ALU、译码器、选择器、地址计算逻辑等。它们的输出只由当前输入决定，但输入变化后，输出需要经过一定传播延迟才会稳定。
2. **时序逻辑电路**：例如 PC、寄存器堆、状态寄存器、流水线寄存器等。它们需要在某个确定的时刻保存新的状态。

如果用时钟电平本身作为控制信号，例如“高电平期间一直允许写入”，就会产生一个问题：只要时钟保持在有效电平，寄存器或存储器就可能持续响应输入变化。这样组合逻辑的中间毛刺、传播延迟、反馈路径中的临时结果，都可能被写入状态单元，造成竞争、冒险或重复写入。

使用**时钟边缘**控制状态更新，核心好处是把状态变化限制在一个很短的瞬间：

- 在两个时钟边缘之间，组合逻辑有足够时间根据当前状态和当前指令计算出下一状态；
- 到达指定边缘时，PC、寄存器等状态部件统一采样并保存稳定后的结果；
- 边缘过后，新的状态再进入组合逻辑，开始下一轮计算。

因此，边缘触发可以把“计算”和“保存”分开，保证处理器按照一个个离散的周期向前推进。

在课程实验或单周期 CPU 设计中，经常会选择**下降沿**作为某些写使能或状态更新的触发时刻。下降沿本身并不比上升沿天然更“正确”，真正重要的是整个系统对时序的统一约定。采用下降沿通常有以下考虑：

1. 给控制信号和数据通路留出稳定时间

如果某些部件在上升沿之后开始输出当前指令、译码结果、ALU 结果和写回数据，那么到下降沿时，这些信号已经经过半个周期的传播，通常更加稳定。此时再进行写寄存器、写存储器等操作，可以降低把未稳定数据写入状态部件的风险。

2. 避免读写同一类状态部件时发生冲突

例如一个周期中需要先读寄存器，再经过 ALU 运算，最后写回寄存器。如果读和写都在同一个电平期间发生，就容易出现“读到新值还是旧值”的歧义。若约定某些读操作在周期前半段完成，写操作在下降沿完成，就能让一次指令执行过程更清晰。

3. 配合上升沿更新 PC 或取指逻辑

有些设计中，上升沿用于更新 PC 或取出新指令，下降沿用于写回当前指令产生的结果。这样一个周期可以被自然分成前后两个阶段：前半段完成取指、译码和运算，后半段保存结果。

4. 避免电平敏感带来的多次触发

若控制信号在高电平期间一直有效，写操作可能在一个周期内被触发多次。边缘触发则只在边沿瞬间响应一次，状态更新更可控。

可以把一个简化的单周期执行过程理解为：

上升沿：PC 更新，开始取当前指令
周期中：指令译码，寄存器读出，ALU 运算，访存或产生写回数据
下降沿：把已经稳定的结果写入寄存器或存储器
下一个周期：继续执行下一条指令

因此，“用下降沿做控制信号”的本质并不是下降沿本身具有特殊功能，而是利用时钟边缘把状态更新固定到一个确定时刻，并通过选择下降沿给组合逻辑留下半个周期的稳定时间，从而减少竞争、冒险和时序不确定性。

二、`_gmon_start_` 的含义

题目中给出的反汇编片段如下：

```
080483e4: push %ebp
080483e5: mov %esp, %ebp
080483e7: sub $0x18,%esp
080483ea: mov 0x8(%ebp), %eax
080483ed: mov %eax,0x4(%esp)
080483f1: lea 0xffffffff0(%ebp), %eax
080483f4: mov %eax, (%esp)
[080483ea-080483f4: 将strcpy的两个实参入栈]
080483f7: call 0x8048330 <_gmon_start_@plt+16>    // what's plt
080483fc: lea 0xffffffff0(%ebp), %eax
080483ff: mov %eax,0x4(%esp)
```

1. 先解释这段代码在做什么

这段代码是典型的 x86 32 位函数序言和函数调用准备过程。

```
080483e4: push %ebp
080483e5: mov %esp, %ebp
080483e7: sub $0x18,%esp
```

这三条指令建立当前函数的栈帧：

- `push %ebp`：保存调用者的栈帧基址；
- `mov %esp, %ebp`：让 `ebp` 成为当前函数栈帧的基址；
- `sub $0x18, %esp`：在栈上分配 0x18 字节的局部空间。

随后：

```
080483ea: mov 0x8(%ebp), %eax
080483ed: mov %eax,0x4(%esp)
080483f1: lea 0xffffffff0(%ebp), %eax
080483f4: mov %eax, (%esp)
```

在 32 位 x86 的 `cdecl` 调用约定中，函数参数通过栈传递。这里的含义是：

- `0x8(%ebp)` 是当前函数的第一个参数；
- `lea 0xffffffff0(%ebp), %eax` 计算局部数组或局部缓冲区的地址，也就是 `ebp - 0x10`；
- `0x4(%esp)` 放第二个参数；
- `(%esp)` 放第一个参数。

如果这次调用是 `strcpy(dest, src)`，那么对应关系可以理解为：

```
char local_buf[...];
strcpy(local_buf, argument);
```

也就是：

```
mov 0x8(%ebp), %eax      ; 取出源字符串参数 src
mov %eax, 0x4(%esp)      ; strcpy 的第二个参数 src
lea -0x10(%ebp), %eax    ; 计算局部缓冲区地址 dest
mov %eax, (%esp)         ; strcpy 的第一个参数 dest
call strcpy
```

2. `_gmon_start_` 是什么

`_gmon_start_` 通常和 GNU 工具链中的 **gprof 性能分析机制** 有关。`gmon` 可以理解为 GNU monitor，用于函数调用计数和采样分析。程序如果使用 `-pg` 等选项进行性能分析编译，运行时会通过相关初始化函数记录函数调用信息，最后生成可供 `gprof` 分析的数据。

在很多 ELF 可执行文件中，即使程序没有真正启用性能分析，也可能在动态链接相关区域看到 `_gmon_start_` 这个符号。它常常以弱符号或动态链接符号的形式出现：

- 如果程序启用了 `gprof`，运行时调用对应的 `profiling` 初始化函数；
- 如果没有启用，对 `_gmon_start_` 的引用可能为空，或者不会产生实际效果；
- 反汇编中看到它，不一定说明用户代码显式调用了 `_gmon_start_`。

所以，`_gmon_start_` 不是程序员在 C 源代码里通常会写的业务函数，而是编译、链接和运行时环境为了支持性能分析和程序初始化而可能引入的符号。

3. 什么是 PLT

题目中特别问到：

```
call 0x8048330 <_gmon_start_@plt+16>
```

这里的 `plt` 是 **Procedure Linkage Table**，即过程链接表。它是 ELF 动态链接机制的一部分。

当程序调用 `printf`、`strcpy`、`malloc` 这类动态库函数时，函数真正的地址通常不在编译期确定，而是在程序加载或第一次调用时由动态链接器解析。PLT 的作用就是给这些外部函数提供一段跳转代码，让程序可以先调用 PLT 入口，再由 PLT/GOT 和动态链接器找到真实函数地址。

和 PLT 配合使用的还有 GOT，即 **Global Offset Table**。简化过程如下：

```
程序调用 strcpy@plt
↓
PLT 入口检查 GOT 中是否已经有 strcpy 的真实地址
↓
如果是第一次调用，跳转到动态链接器解析 strcpy 地址
↓
动态链接器把真实地址写入 GOT
↓
以后再次调用 strcpy@plt 时，直接通过 GOT 跳到 libc 中的 strcpy
```

这种机制称为**延迟绑定**。它的优点是程序启动时不必立刻解析所有动态库函数地址，只有函数真正被调用时才解析，从而减少启动开销。

4. 为什么显示成 `<_gmon_start_@plt+16>`

`<_gmon_start_@plt+16>` 并不一定表示调用的是 `_gmon_start_` 本身。它表示目标地址位于 `_gmon_start_@plt` 这个符号之后偏移 16 字节的位置。

在 i386 ELF 中，一个 PLT 表项通常是 16 字节。假设：

```
0x8048320: _gmon_start_@plt
0x8048330: 下一个 PLT 表项
```

那么 `0x8048330` 就刚好是 `_gmon_start_@plt + 16`。反汇编器如果没有在这个地址显示更准确的符号名，或者符号信息不完整，就可能把它标注成“距离最近的已知符号加偏移”的形式。

结合前面四条传参指令可知，这里已经准备好了 `strcpy` 的两个参数，因此该 `call` 很可能实际是在调用 `strcpy@plt`，只是反汇编显示时用 `_gmon_start_@plt+16` 这种形式标出了地址位置。

因此，这条指令可以理解为：

```
call strcpy@plt
```

只是反汇编输出中把它写成了：

```
call 0x8048330 <_gmon_start_@plt+16>
```

5. 后续 `printf` 参数准备

在 `strcpy` 返回后：

```
080483fc: lea 0xfffffffff0(%ebp), %eax
080483ff: mov %eax, 0x4(%esp)
```

这里又把局部缓冲区地址作为某个函数调用的参数放到栈上。题目中说“将 `printf` 的两个实参入栈”，说明后续代码大概率还会把格式字符串地址放入 `(%esp)`，把局部字符串地址放入 `0x4(%esp)`，然后调用 `printf@plt`。

也就是说，这段程序的大致逻辑可能是：

```
void f(char *s) {
    char buf[...];
    strcpy(buf, s);
    printf("...", buf);
}
```

总结来说：

- `_gmon_start_` 是 GNU profiling 相关的初始化符号；
- `PLT` 是动态链接时用来调用外部库函数的跳转表；
- `@plt+16` 说明目标地址是某个 PLT 表项之后 16 字节的位置；

- 结合传参过程，这里的调用更应该从程序语义上理解为调用 `strcpy@plt`。

三、为什么 `ori` 指令是 0 扩展，而 `load` 是符号扩展？

这个问题的关键在于：**立即数扩展方式取决于指令语义**。不同指令对 16 位立即数的解释并不相同，因此硬件不能简单地对所有立即数都采用同一种扩展方式。

1. `ori` 是逻辑运算，立即数应被看作位模式

`ori` 是 bitwise OR immediate，即“按位或立即数”。它的立即数不是一个有符号整数，而是一个用于按位运算的二进制掩码。

例如：

```
ori $t0, $zero, 0xffff
```

这条指令通常希望得到：

```
$t0 = 0x0000ffff
```

如果对 `0xffff` 做符号扩展，结果会变成：

```
0xffffffff
```

这样就完全改变了逻辑或的掩码含义。

再比如，MIPS 中常用 `lui` 和 `ori` 拼接 32 位常数：

```
lui $t0, 0x1234
ori $t0, $t0, 0x5678
```

预期结果是：

```
$t0 = 0x12345678
```

如果 `ori` 的立即数采用符号扩展，那么当低 16 位最高位为 1 时，就会把高 16 位也扩展成 1，从而破坏 `lui` 已经放入高 16 位的内容。比如：

```
lui $t0, 0x1234
ori $t0, $t0, 0x8000
```

正确结果应为：

```
$t0 = 0x12348000
```

如果 `0x8000` 被符号扩展成 `0xffff8000`，按位或后会得到：

```
$t0 = 0xffff8000
```

这显然不是构造常数时想要的结果。

因此，`ori` 必须采用 0 扩展。它保证 16 位立即数只影响低 16 位，高 16 位补 0，从而符合逻辑掩码和常数拼接的语义。

2. load 的符号扩展通常来自两类语义

“load 是符号扩展”需要分情况理解，因为 load 指令中可能涉及两种不同的扩展：

1. 地址偏移量的符号扩展
2. 加载较小数据类型后的符号扩展

2.1 地址偏移量为什么要符号扩展

很多 load 指令采用“基址寄存器 + 立即数偏移”的寻址方式，例如：

```
lw $t0, -4($sp)
lw $t1, 8($fp)
```

这里的 `-4`、`8` 是地址偏移量。偏移量显然需要能够表示正数，也需要能够表示负数：

- 正偏移可以访问结构体后面的字段、数组后面的元素；
- 负偏移可以访问栈帧中的局部变量、保存的寄存器或调用参数。

如果偏移量采用 0 扩展，那么：

```
lw $t0, -4($sp)
```

中的 `-4` 在 16 位补码中是：

```
0xffffc
```

若 0 扩展，则变成：

```
0x0000ffff = 65532
```

这会让地址变成：

```
$sp + 65532
```

而不是预期的：

```
$sp - 4
```

因此，对于 load/store 中的地址偏移量，符号扩展是必要的。它让一条指令可以通过有限的 16 位立即数表示正负两个方向的地址偏移。

2.2 加载 byte/halfword 后为什么有符号扩展

另一种情况是从内存中加载小于寄存器宽度的数据。例如 MIPS 中有：

```
lb    $t0, 0($a0)    # load byte, 有符号扩展
lbu   $t0, 0($a0)    # load byte unsigned, 0 扩展
lh    $t0, 0($a0)    # load halfword, 有符号扩展
lhu   $t0, 0($a0)    # load halfword unsigned, 0 扩展
lw    $t0, 0($a0)    # load word, 加载整字, 不需要扩展
```

如果内存中有一个字节：

```
0xff
```

那么：

- `lb` 把它当作有符号 8 位整数 `-1`，扩展到 32 位后得到 `0xffffffff`；
- `lbu` 把它当作无符号 8 位整数 `255`，扩展到 32 位后得到 `0x000000ff`。

所以，不是所有 load 都一定符号扩展，而是：

- 有符号加载指令，例如 `lb`、`lh`，需要符号扩展；
- 无符号加载指令，例如 `lbu`、`lhu`，需要 0 扩展；
- 加载完整字长的 `lw` 不涉及扩展；
- load/store 的地址偏移量通常需要符号扩展。

3. `ori` 和 load 的本质区别

`ori` 的立即数表示“位掩码”，所以必须 0 扩展：

```
ori: 16 位立即数表示低 16 位 bit pattern, 高 16 位补 0
```

load 的偏移量表示“有方向的地址位移”，或者加载的数据本身可能表示“有符号小整数”，所以需要符号扩展：

```
load offset: 16 位立即数表示有符号位移，可以向前或向后访问内存
lb/lh: 内存中的 byte/halfword 表示有符号整数，需要保留符号
```

因此，扩展方式不是由“立即数都是 16 位”这个事实决定的，而是由指令对立即数或数据的解释方式决定的。逻辑运算需要保持位模式，地址偏移和有符号小整数需要保留正负号，这就是 `ori` 使用 0 扩展、load 相关场景常使用符号扩展的原因。